

Building CLIs for AI Agents Cheat Sheet

Irurueta Docs › Guides & References › AI › Building CLIs for AI Agents

Baseline (checked 2026-09-30): Python 3.12 · Typer 0.27.2 · Click 8.5.0 · Rich 15.0.0 · Java 21 · picocli 4.7.7 · Node.js 24 · commander 15.0.0 · oclif 6.0.2 · Go 1.27.1 · Cobra v1.10.2 · clap 4.6.7 · Rataui 0.30.2 · Rig 0.42.0 · Swift 6.4 · swift-argument-parser 1.8.2 · System.CommandLine 2.0.12 · FastMCP 4.0.10 · @modelcontextprotocol/server 2.2.0 · Spring Boot 4.1.1 + Spring AI 2.0.1 · MCP spec 2026-07-28. The tool is the fictional docsctl 0.1.0; tokens are always \$DOCSCTL_TOKEN, never in code. No benchmark numbers or prices. Dated 2026-09-30.

DESIGN RULES

- **Three audiences, one tool:** humans, scripts, agents; same contract
- POSIX: options before operands, -- ends options, -- = stdin; GNU: --long, --help, --version
- Noun-verb or verb subcommands, one naming style; same flag = same meaning in every command
- Flags over positionals when more than one; no ambiguous prefix matching or aliases that shift
- 12-factor CLI: config by env, stdout/stderr split, exit codes, no side effects on --help
- cli.dev: be human-first *and* machine-readable; be explicit about what changes

Commands: search show list- components validate publish login logout schema completion

STREAMS RULE

- **stdout = data only** (text, JSON, JSONL); **stderr = everything else:** logs, progress, prompts, error text
 - Pipes must survive: handle SIGPIPE, callers use set -o pipefail
 - Crash-only: exit fast, no cleanup that can hang; the next run recovers
- ```
docsctl search kalman --json
| jq -r '.results[]|id'
docsctl publish --component c --json \
 2>err.json; echo $?
```

EXIT CODES (DOCSCTL)

| Code      | Name              | Meaning                                  |
|-----------|-------------------|------------------------------------------|
| 0         | OK                | success, idempotent no-op, dry-run plan  |
| 1         | INTERNAL          | unexpected failure                       |
| 2         | USAGE             | bad flag or argument                     |
| 3         | VALIDATION        | input rejected; validate found problems  |
| 4         | NOT_FOUND         | component or page missing                |
| 5         | CONFIRM_REQ UIRED | envelope holds the exact confirm command |
| 6         | RETRYABLE         | rate limit, 5xx, timeout                 |
| 7         | AUTH              | missing or invalid credentials           |
| 126 / 127 | shell             | not executable / not found               |
| 128+ N    | signal            | 130 SIGINT, 143 SIGTERM                  |

Custom codes only in 3-63. sysexits (64-78) is vocabulary stay; it is deprecated on FreeBSD.

TTY, COLOUR, INTERACTIVITY

- Prompt only when stdin *and* stderr are TTYs and --no-input / DOCSCTL\_NO\_INPUT are unset
- Otherwise fail fast: exit 5 (confirm) or 2 (missing arg) and **name the bypass flag** (--yes, --force)
- Colour off when: not a TTY, NO\_COLOR set, TERM=dumb, or --no-color
- Spinners and progress bars: stderr, TTY only
- Detect with isatty(3) (Python sys.stdout.isatty(), Java System.console().isOutputRedirected)
- Windows: enable virtual-terminal sequences before emitting ANSI

CONFIG PRECEDENCE AND AUTH

- ```
flags > env DOCSCTL_*
> project ./docsctl.toml
> user $XDG_CONFIG_HOME/docsctl/config.toml
> system /etc/docsctl/config.toml
```
- User dir: Linux ~/.config/docsctl; macOS ~/Library/Application Support/docsctl; Windows %APPDATA%\docsctl
 - Token order: DOCSCTL_TOKEN > DOCSCTL_TOKEN_FILE > keychain > credentials file (0600)
 - Headless: docsctl login --token-stdin; no browser flow required
 - Env vs. files for secrets is a trade-off (cli.dev vs. agent-CLI posts), not a verdict

JSON OUTPUT CONTRACT

- ```
{"schemaVersion":1,
"query":"kalman","total":2,
"nextCursor":null,"truncated":false,
"results":[{"
 "id":"irurueta-
statistics:kalman.adoc",
 "title":"Kalman
filter","score":0.91}]}
```
- stderr:** {"schemaVersion":1,"error":{"code":"AUTH\_REQUIRED","exitCode":7,"message":"No credentials found.", "hint":"Run 'docsctl login' or set DOCSCTL\_TOKEN.", "retryable":false}}
- --json = --format json|jsonl|text; JSONL: one object per line + final summary line
  - --fields a,b, --jq EXPR; sorted keys, deterministic order
  - schemaVersion, additive-only changes; publish JSON Schemas
  - Text output is for eyes only, never parsed

AGENT ERGONOMICS

- Small defaults: --limit 10 (max 100), --cursor; truncated marker
- Concise by default, --max-chars / detail on request; semantic IDs (component::page)
- Errors say what to do next (hint, exact command)
- --timeout 30; never hang on input
- Fast start: native binary (GraalVM, Go, Rust)
- Claude Code caps MCP tool results at 25,000 tokens by default (warns at 10,000)
- Rule of thumb: T ≈ N · c / 4 tokens (N results, c characters each, ~4 characters per token); measure with your model's counter

SAFETY PATTERN

- ```
docsctl publish --component c
--dry-run --json
→ {"dryRun":true,
  "plan":{
    "add":3,"change":1,"remove":0},
  "planId":"pln_...", "confirm":{"required":true,"command":
  "docsctl publish --component c
  --confirm-token pln_... --json"}}
```
- docsctl publish ... --confirm-token pln_...
→ exit 0
no token, no TTY → **exit 5** + envelope
- Read-only by default; dry-run emits a JSON plan; --force is explicit
 - Idempotent create-or-update with --idempotency-key
 - Undo / rollback hint in the result
 - Confirmation stays with a person, not the wrapper

CLI VS. MCP

	CLI via shell	MCP server
Context	nothing loaded until run	schemas loaded up front
Discovery	--help, known tools	tools/list
Typing	flags, JSON out	JSON Schema in and out
Auth	env, files, keychain	OAuth 2.1 (remote)
Govern.	host permissions, sandbox	annotations, gateways
Hosts	need a shell	any MCP host

Ship both when you can (gh + GitHub MCP server). Token comparisons from Firecrawl and CircleCI are vendor claims.

WRAP AS AN MCP SERVER

- Python** FastMCP
@mcp.tool(annotations={ "readOnlyHint": True, "idempotentHint":True}, output_schema=OUT)
def docsctl_search(query, limit=10): subprocess.run([DOCSCTL,"search", "--json",,...,"-",query], shell=False, timeout=25, env=MIN_ENV, stdin=subprocess.DEVNULL)
TypeScript
@modelcontextprotocol/server
server.registerTool("docsctl_search", {inputSchema, outputSchema, annotations}, handler);
execFile(DOCSCTL, argv, {timeout:25000, maxBuffer:256000, env, shell:false})
Spring AI @McpTool
@McpTool(name="docsctl_search", generateOutputSchema=true, annotations=@McpTool.McpAnnotations(readOnlyHint=true))
new ProcessBuilder(argv);
p.waitFor(25, TimeUnit.SECONDS)
- Exit code ≠ 0 → isError; JSON stdout → structuredContent; schema → outputSchema. Agent tools: LangChain @tool, Spring AI @Tool, LangChain4j @Tool; Claude Code Bash(docsctl *).

INJECTION DEFENCE

- **argv arrays, never shell strings** (CWE-88, OWASP)
- -- before every user operand
- Allowlist subcommands and flags; regex-validate names
- Reject values starting with -
- Absolute binary path; minimal environment; stdin closed
- Least-privilege credentials; sandbox (container, Claude Code sandbox)
- Cap output size and sanitise it: output is untrusted text for the model
- Sign releases; publish checksums

TESTING CHECKLIST

- Golden test of --json output = the contract
- No TTY means no prompt (exit 5 or 2, bypass flag named)
- One test per exit code; --help snapshots
- bats-core (shell), Click/Typer CliRunner, picocli setOut/setErr, trycmd / insta (Rust), testscript (Go)
- hyperfine for start-up time
- Agent evals: an LLM completes tasks with the CLI

FRAMEWORKS PER LANGUAGE

Lang.	Use
Python	Typer (Click, argparse); Rich, Textual
Java	picocli + picocli-codegen → GraalVM native-image; Spring Shell; JLine
Node.js	commander , yargs, oclif; Ink
Go	Cobra ; Bubble Tea
Rust	clap (clap_complete, clap_mangen); Rataui; Rig
Swift	swift-argument-parser ; SwiftTUI (dormant)
.NET	System.CommandLine ; Spectre.Console(.Cli)

A TUI is a view over the command line; every action stays reachable non-interactively.

PACKAGING TABLE

Channel	Install
pipx / uv	pipx install docsctl; uv tool install
Homebrew	brew install OWNER/tap/docsctl
npm	bin field + shebang; npx
JBang	jbang app install
Native binary	download, verify, PATH
cargo	cargo install --locked
winget	manifest in community repo
.NET tool	dotnet tool install --global
Container	docker run --rm IMAGE; pin by digest

GoReleaser builds archives, checksums and signatures; sign with Sigstore cosign or GitHub attestations.

EXIT CODES AND DOCS

- picocli: exitCodeOnInvalidInput, IExitCodeGenerator
- Typer: raise typer.Exit(code)
- Docs from the tree: picocli man pages, clap_mangen, Cobra doc, Ascidoctor manpage backend

SELF-DESCRIBING HELP

- --help: examples first, every flag explained
- docsctl schema --json: command tree, flags, output schema, exit codes, safety metadata
- docsctl completion bash|zsh|fish|powershell; man pages from the tree
- Ship a SKILL.md agent skill next to the CLI
- Generate help, docs and MCP schemas from one source

AGENT-READINESS IN TEN LINES

- JSON on stdout, schema versioned
- No prompts without a TTY
- Exit codes documented and tested
- Dry-run and confirmation for destructive ops
- Small, paginated output
- Errors with remediation
- Timeouts everywhere
- schema --json and examples in help
- Headless auth, placeholder secrets
- Config precedence documented